I have been in the mode of writing down technical details for all of the things that we have done. I have not been in the frame of mind for argumentative and persuasive writing. Why should people care about the paper?

Proof formalization has several great strengths: we gain a high degree of certainty in our proofs, the computer can perform bookkeeping of the intracacies found when splitting into many cases, and a consumer of a proof can treat it as a black box provided if they trust the interface/signature of a proof. Thus formalization artifacts offer a great opportunity to remove the fallibility of the human element of proof checking, which can [illegible] be a strength during the review process.

However, the opaque [illegible] terse nature of formalization can also be its biggest weakness. Formal proofs are complex and hard to understand by a reader. Perhaps you ask yourself if there are, [illegible] if there ought to be, any readers of formal proofs? A negative response to this question is primarily a reflection of the current state of affairs rather than an ideal to strive towards. Surely it is preferrable for an audience, no matter how small, to be able to digest the content produced.

Much more compelling than optimizing the consumption of formal proofs is the optimization of [illegible] of formal proofs.

In the same way that assembly code is harder for a developer then programs in a high level language, it too is harder to read/write low level formal proofs. When doing proofs on paper, the practitioner can freely invoke high level reasoning principles or cast between equivalent formal presentations with no issues. However, we are not afforded those liberties when doing mathematics within a proof assistant like Agda. Each choice when implementing a mathematical construct within dependent type [illegible] commits to a particular presentation of a mathematical idea, and not all presentations are created equal. The proof developer should take great care in choosing the presentation that is maximally beneficial to their desired goal.

So are we done then. Given a proof target we simply choose the "best" presentation and call it a day? Well no. Even if we knew what our selection criteria should be — which is an [illegible]able question — there is still a fundamental issue. Certain presentations of a given theory (used very loosely) may each [illegible] optimal for proofs of different theorems, and we usually want to prove more than just a single theorem.

So what are we to [illegible] then? How can we abstract over [illegible] like a choice of presentation in a con[illegible] enough manner to still be useful for formalization? There is an answer to this question and we have known it as a community for decades. It's category theory!

However, this too is still not a total solution for our formalization goals. On paper, category theory is a powerful tool that we use [illegible] the time when abstracting within mathematics, and particularly in programming language theory.

There is a wealth of applications of category [illegible] in describing the semantics of programming languages. For instance, several metatheoretic results about PL semantics — canonicity, parametricity, normalization, etc — are encapsulated and proved quite succinctly in the language of category theory.

However, even these results don't always translate nicely to formalization. For instance, consider a canonicity logical relation can be captured using ideas from the theory of Grothendieck fibrations. However, there is a large footgun hidden within the usual presentation of Grothendieck fibrations that [illegible] complications when formalizing them within a proof assistant. Importantly, the definition mentions a propositional equality, see Ahrens and Lumsdaine for details. Therefore when reasoning about fibrations in Agda we will naturally need to transport terms across these equalities, landing us in transport hell.

There is a clever work[illegible] to the deficiencies of the classical presentation of Grothendieck fibrations. Ahrens and Lumsdaine introduced the theory of displayed categories.

In short, functors (and thereby fibrations) can define a family of fiber categories over an element of the codomain. i.e. For $F : C \to D$, $d \in D$, we can define a category of elements of C that map to d under F. A fibration is a functor $C \to D$ for which these fiber categories vary functorially (contravariantly) in d.

The equality condition mentioned above stems from constraints like $F c \equiv d$ describing these fibers.

Displayed categories offer a repackaging of this data such that $F c \equiv d$ is an equality that is guaranteed to hold definitionally and thus can be elided. In some sense they are a strictification of the families of fibers induced by a functor.

On paper the distinction between a displayed category and a functor is immaterial, as they are equivalent mathematically. However, this distinction in definitional behavior is of the utmost importance when designing a formalization project.

(Displayed) category theory then gives a suitable target for choice of abstraction within the proof assistant when doing formalization of programming language metatheory and we demonstrate this through the implementation of logical relations for canonicity, conservativity, and the coherence theorem for monoidal categories.

Across each of these proofs we are able to reuse constructions such as

- reindexing of a displayed category
  - I think that this is a suitable candidate for "gluing"
- global/local sections
- (displayed) universal properties and their preservation under reindexing

As a sketch of the framework

"Gluing for Type Theory" says that canonicity is *just* what you get when gluing along the global sections functor. However, in my experience that level of abstraction doesn't readily translate to how we would conduct a proof of canonicity within the proof assistant.

However, we can make this literally true in Agda. Of course you pay the price of frontloading work when making the library, but after you have put in this work it gives for succinct instantiations.

Canonicity for the free cartesian category

We define the free cartesian category over a signature. It has the universal property that you may expect given its name.

For any cartesian category C w/ an interpretation $\iota : \Sigma \rightsquigarrow C$, we can define a cartesian functor $rec : CC[\Sigma] \to C$

Using displayed categories we can state a slightly different universal property:

For any displayed cartesian category C^D over $CC[\Sigma]$ w/ $\iota^D : \Sigma \rightsquigarrow C^D$, we can define a global section

$$\begin{tikzcd} C^D \arrow[d] \\ CC[\Sigma] \arrow[u, dashed, bend right, "elim"'] \end{tikzcd}$$

Displayed categories are to categories as dependent types are to types.

We think of this global section like a dependent elimination principle and the functor rec as a simply-typed recursor

As a note to self, I think that we can define the eliminator from the recursor.

- Can't we say that $C \to \int D^D$ is the same data as a section $\begin{tikzcd} & D^D \arrow[d] \\ C \arrow[ur] \arrow[r] & D \end{tikzcd}$

we have an operation $(\text{section } F\ C^D) \to \text{Functor } D\ \int C^D$ called intro

And in the other direction,

$\text{Functor } D\ \int C^D \to \text{Section } F$

$$\begin{tikzcd} & & C^D \arrow[d] \\ D \arrow[r] & \int C^D \arrow[ur, "snd"] \arrow[r, "fst"] & C \end{tikzcd}$$

In particular,

$$\begin{tikzcd} & & C^D \arrow[d] \\ CC[\Sigma] \arrow[r, "rec"] & \int C^D \arrow[ur, "snd"] \arrow[r, "fst"] & C \end{tikzcd}$$

fst is Cartesian, so is rec.

Thus rec ∗ fst is the unique Cartesian functor $CC[\Sigma] \to C$

Further, we can likely argue that snd is a cartesian section?

IDK if this is even relevant anymore

Wrapping up the canonicity example, we can use reindexing (i.e. gluing) to get a displayed cartesian category over $CC[\Sigma]$ for which we will build a section using the universal property.

$$\begin{tikzcd} \text{reindex } Set^D \arrow[r] \arrow[d] \arrow[dr, phantom, "\lrcorner", very near start] & Set^D \arrow[d] \\ CC[\Sigma] \arrow[r, "Cl(-)"] & Set \end{tikzcd}$$

Because reindex Set^D is displayed cartesian closed, we may construct a section by interpreting our signature Σ

Importantly, we interpret Σ in a manner that we interpret each base type as a proof that the closed term $\cdot \vdash M : A$ is in canonical form.

If Σ is just Bool, then

$\iota\ Bool : Cl(Bool) \to Set$

$\iota\ Bool\ e := e \equiv t \uplus e \equiv f$

Then when interpreting terms t and f, we provide a witness to being canonical via inl(refl) and inr(refl)

That's all to say, we leverage precisely the same high-level abstractions that we are able to when doing the paper proof.

Category theory is an adequate and general abstraction for proofs of this kind. Importantly, once the category theoretic principles were implemented, this proof is just as easy as it is on paper

- And it is readable
- And it makes no concession on postulates
- And it is a relatively flexible framework

The points where a user has choice is in swapping out their free category, and they have the obligation to prove the appropriate universal property. They must also demonstrate the preservation of this universal property under reindexing

- This bit isn't as free as I would hope, but perhaps what Max has recently wrote makes this tractable
- And maybe there is an appeal to the relative infrequency of needing a new universal categorical structure
- Or even the opportunity for using 2 category theory to get these "for free"

There are many bits that are reusable and modular but there is a decent amount of work in adding a new univprop / adding a new theory. I want to be transparent about that caveat

I think that this suggests a framework for library design rather than a universal solution

I can demonstrate the compositional and reusable nature of the formalization

I need to highlight that transport hell is not only tedious, but that it can also be a source of performance woes. Normalization of incredibly large paths, even in a homotopy set, was a large roadblock when working on this project.

Our solution was to treat paths as complementary to an inductive equality type rather than a replacement for them

- Eliminating an equality which is Eq-refl preserves nice definitional behavior, whereas subst/transport are not definitionally the identity when applied to refl

Must also write about reindexing theorems and local-to-global principles

In summary, the thesis of this work is that displayed category theory provides a strategy for encapsulating category theory within dependent type theory in a reusable and compositional manner. In the same vein, cubical type theory is the appropriate setting for formalization because notions of heterogeneous equality arise naturally within the definition of a displayed category / within the theory of fibrations more broadly.